

PPC Audit & Profit Console

Comprehensive Code Documentation

Amazon PPC audit + profit console.
FastAPI · SQLAlchemy · pandas | React (Vite) · Tailwind
100% local · no Amazon Ads API

Generated 2026-07-24 · ~18,700 LOC backend · 140+ source files

Contents

1.	System Overview
2.	Technology Stack & Dependencies
3.	Architecture at a Glance
4.	Authentication & User Isolation
5.	Multi-Tenant Data Model (store → project → cadence)
6.	Database Spine (database.py)
7.	Schema — Star Model & Side Tables
8.	Metrics & Bid Math
9.	Pipeline Stages & Flag Engine
10.	Audit Cadences (5 workflows)
11.	Machine Learning Module (ml.py)
12.	Account-Level Engines
13.	Store-Level: Catalog, Transactions, Listing Optimizer
14.	Bulk File Safety & Parsers
15.	API Reference (all endpoints)
16.	Frontend Architecture
17.	Design System
18.	Development, Testing & Conventions

1. System Overview

The app is an Amazon PPC (Pay-Per-Click) audit and profit console. A user uploads Amazon advertising exports (bulk files, search-term reports, business reports, payments reports) and the app produces: flags, bid plans, profit P&L, keyword research, and executive reports.

Core principle: 100% local, no Amazon Ads API

The app never calls Amazon programmatically. Every "action" the app recommends becomes a downloadable **bulk .xlsx** file that the user re-uploads to Amazon Seller Central manually. This shapes the whole design — the app must emit files that pass Amazon's bulk validation exactly (see §14).

What it does, end to end

- **Ingest** — Amazon exports parsed into a star schema (per user, per store, per audit, per cadence).
- **Audit** — a pure-function flag engine scores every target/keyword against tunable thresholds.
- **Optimize** — bid optimizer, placement optimizer, search-term harvest, n-gram mining.
- **Profit** — P&L from ad cost + COGS + Amazon fees + referral/refund; break-even ACoS per ASIN.
- **Cadences** — 5 distinct recurring audit workflows (Daily / Weekly / Mid-Month / Full-Month / Pause-Scale), each with its own data and its own emitted bulk.
- **Account engines** — Waterfall restructure, Cannibalization detector, SB/SD channels.
- **Listing / SEO** — keyword research + a competitor/SEO rank tracker (Listing Optimizer).
- **Report** — executive summaries + downloadable exports; optional LLM narration.

Standing rule (from CLAUDE.md): any feature added/changed must update **MANUAL.md** (the user guide) in the same change — the relevant section plus the Changelog list at the bottom.

2. Technology Stack & Dependencies

Backend (Python, from `backend/`)

Package	Version	Role
fastapi	0.115.0	HTTP API, dependency injection, routers
uvicorn[standard]	0.30.6	ASGI server (<code>--port 8000</code>)
sqlalchemy	2.0.34	ORM + per-file SQLite engines
pydantic	2.9.2	Request/response schemas (<code>schemas.py</code>)
pandas	2.2.3	Parsing + aggregation of exports
numpy	2.x	Hand-rolled ML (<code>ml.py</code>), no <code>scipy/sklearn</code>
openpyxl	3.1.5	xlsx read fallback + all bulk writes
python-calamine	0.8.2	Fast Rust xlsx reader (~8x <code>openpyxl</code>); optional
python-multipart	0.0.9	File uploads
httpx	0.28.1	Tests only (FastAPI <code>TestClient</code>)

No `scipy`, no `scikit-learn`, no LaTeX — the ML module (§11) is NumPy-only and deterministic. LangChain is optional, only when `LLM_USE_LANGCHAIN=true`.

Frontend (React, from `frontend/`)

Package	Version	Role
react / react-dom	18.3.1	UI
vite	5.4.8	Dev server (:5173, proxies <code>/api→:8000</code>) + build
tailwindcss	3.4.13	Token-driven design system
chart.js + react-chartjs-2	4.5 / 5.3	Only chart deps (Monitoring tab)

3. Architecture at a Glance

```
Amazon exports (.xlsx/.xlsm/.csv)
  | upload
  v
FastAPI routers (backend/app/routers/*.py) -- auth-gated, ?store=&project=&audit_type=
  | Depends(get_db) -> per-user/store/project/cadence SQLite file
  v
Pipeline (backend/app/pipeline/*.py)
  ingest -> split -> clean -> load          (build star schema)
  audit (rules.py flag engine)              (score targets)
  side pipelines: harvest, ngrams, sales, profit, benchmark,
                  bidopt, placement, strategy, keywords, report...
                  | emits validated Amazon bulk .xlsx (bulkfmt.py) + logs to ChangeLog
                  v
React SPA (frontend/src) – tabbed shell, one DataTable everywhere,
  fetches /api with Bearer token, ?store/project/audit_type on every call
```

Layer responsibilities

Layer	Files	Responsibility
Config	<code>config.py</code>	Settings (env-driven): CORS, superuser, LLM provider
Auth	<code>auth.py</code> , <code>routers/auth.py</code>	Central auth DB, tokens, user CRUD
DB spine	<code>database.py</code>	Per-file engine cache, scoping, <code>get_db</code> dependency
Schema	<code>models.py</code> , <code>schemas.py</code>	SQLAlchemy tables + Pydantic I/O
Metrics	<code>metrics.py</code> , <code>rules.py</code>	Pure metric + flag functions
Pipeline	<code>pipeline/*.py</code>	Staged ingestion + all analytics engines
Routers	<code>routers/*.py</code>	Thin HTTP wrappers over pipeline
Frontend	<code>frontend/src</code>	React tabbed SPA

4. Authentication & User Isolation

Central auth store — `auth.py`

- Own engine/Base, separate DB file `data/auth.db` — NOT part of the per-store pipeline DBs.
- Username/password with stdlib **PBKDF2** hashing: format `pbkdf2_sha256$iters$salt$hash` — no native crypto dependency.
- Login mints an opaque bearer token, stored server-side in a `sessions` table, sliding 12h expiry.
- **One active session per user** — a new login revokes prior sessions.
- Superuser `SAdmin / RootPass` seeded on startup by `init_auth()`; override via `SUPERUSER_NAME / SUPERUSER_PASS` env.

Routes — `routers/auth.py`

Endpoint	Access	Purpose
POST <code>/auth/register</code>	public	Self-signup → normal user + auto-login (min 6-char pw, no email/OTP)
POST <code>/auth/login</code>	public	Mint token (revokes prior sessions)
POST <code>/auth/logout</code>	token	Revoke session
GET <code>/auth/me</code>	token	Current user
GET POST DELETE <code>/auth/users...</code>	superuser	User CRUD

Per-user data isolation

Each store directory is namespaced by the owner's numeric user id. Two id forms:

- **Internal id**: `u<id>__<store>` (on disk).
- **Public id** (API/UI): un-prefixed slug.

`database.scoped(user.id, store)` translates at the boundary: `get_db` namespaces transparently, so every `Depends(get_db)` router is per-user isolated with zero extra code. New users auto-get an isolated `zvalves / "My Store"`. User ids never reuse (`sqlite_autoincrement`); `delete_user` purges owner store dirs so a recycled id can't inherit stale data. The bundled sample seed lives in the **SAdmin** namespace.

In `main.py`, every data router is gated `dependencies=[Depends(auth.current_user)]`. Only `/auth/*` and `/health` are public. On the frontend, `api/client.js` keeps the token in `localStorage`, attaches `Authorization: Bearer` to every request via `authFetch`, and on any 401 clears it + fires `ppc-unauthorized`. `App.jsx` gates the whole UI behind `Login.jsx`; superusers get a **Users** tab.

Scales to 1000+ users (indexed rows). Swap SQLite→Postgres by changing the URL.

5. Multi-Tenant Data Model

This is the core thing to understand. Data is isolated at **three physical levels**, each a separate SQLite file. Campaign IDs from different Amazon accounts can never collide because the files are physically separate.

```
backend/data/stores/<store_id>/<project_id>[__<cadence>].db

user  -> store      -> project (audit)  -> cadence
(u<id> prefix) (dir)   (base .db file)  (__<cadence> suffix)
```

Level 1 — Store (per user)

Directory namespaced by user id. Store-wide JSON side files live here:

File	Holds
<code>_meta.json</code>	Titles, per-audit settings (Goal ACoS, refund/referral/COGS %), brand terms, upload meta
<code>_benchmark.json</code>	Store-wide Product Benchmark (upload once, every audit matches)
<code>_costs.json</code>	Product Costs (COGS) — store-level, shared across cadences
<code>_catalog.json</code>	Product catalog (Category Listings Reports)
<code>_transactions.json</code>	SKU transaction ledger (Payments report)
<code>_fba.json</code>	Base fulfillment fee + referral fee per unit (Selling economics / FBA Fee Preview)

Level 2 — Project / Audit

Each audit is its own base `.db`. Titles + per-audit settings live in the store's `_meta.json`.

Level 3 — Cadence (per-cadence isolation)

Each Audit Cadence gets its **own db file** `<project>__<cadence>.db`:

Cadence	DB file
daily / weekly / mid_month / pause_scale	<code><project>__<cadence>.db</code>
full_month (and "no cadence")	<code><project>.db</code> (base, back-compat)

So a PPC bulk uploaded under Weekly **only** audits Weekly — its campaigns/flags/bid-opt/harvest/negatives never overlap another cadence. This needed **zero pipeline changes**: `get_db` folds the cadence into the db path (`_data_pid`), so every `Depends(get_db)` router isolates per cadence automatically — exactly like per-user namespacing did for stores.

Shared across all cadences of a project (so you don't re-enter per cadence): Goal ACoS + econ (`_meta.json`, keyed by *base* project), Product Benchmark (`_benchmark.json`), Product Costs (`_costs.json`). Monitoring uses `get_base_db` (always the base file, cadence-agnostic). `CADENCE_KEYS` is hardcoded in `database.py` to avoid an import cycle.

6. Database Spine — `database.py`

The single most important backend file. It turns query params into the correct physical SQLite file and hands routers a scoped session — so no router carries scoping code.

- `get_db()` / `get_session()` read `?store=` & `?project=` query params and return a session for that file. As a FastAPI dependency this makes **every pipeline router auto-scoped, zero per-router code** — new routers just declare `Depends(get_db)`.
- The session carries `(store, project)` in `session.info` (plus `cadence` / `data_project`) so pipelines reach store-scoped resources (store-wide benchmark + costs) without extra params.
- `get_db` also reads `?audit_type=` (the cadence) and scopes the db file to it. `get_base_db` is the same dependency but pinned to the base file (for cross-cadence features like Monitoring).
- `pinned_db(cadence)` = a factory pinning a router to ONE cadence, ignoring the ambient `?audit_type=`. The five cadence routers use it (their URL path *is* the cadence), so background keep-alive panels always hit their own db file.
- Engines are cached lazily in `_engines`. `slug()` sanitizes ids and **blocks path traversal**.
- `delete_store` / `delete_project` dispose engines before removing files. The UI switches selection away first so an in-flight request doesn't re-create a file via `get_db` auto-create.
- SQLite FK enforcement is ON (per-connection pragma). The loader sets `PRAGMA defer_foreign_keys=ON` so dims can insert in any order.

Generic KV helpers

Helper	Scope
<code>get_store_extra</code> / <code>set_store_extra</code>	store <code>_meta.json</code> keys (brand terms, upload meta)
<code>get_project_extra</code> / <code>set_project_extra</code>	per-project keys (waterfall settings, consult result)
<code>get_store_costs</code> / <code>set_store_costs</code>	store <code>_costs.json</code>

7. Schema — Star Model & Side Tables

Defined in `models.py`. Star schema: dimensions hold slow-changing structure; one fact table holds raw metrics stamped with `snapshot_date` (grows each upload → enables trends + ML).

Star schema — dimensions + central fact

Table	Grain / notes
<code>dim_product</code>	ASIN (PK), sku, title, optional <code>break_even_acos</code> margin override
<code>dim_campaign</code>	<code>campaign_id</code> (PK), <code>targeting_type</code> auto manual, state, <code>daily_budget</code> , <code>portfolio_id</code>
<code>dim_ad_group</code>	<code>ad_group_id</code> (PK), FK campaign, <code>default_bid</code> , state
<code>dim_ad</code>	the ad row (ASIN/SKU); view re-roots the tree at the ad's ASIN
<code>dim_target</code>	keyword / product-target; also stores existing negatives (<code>negative_keyword</code> / <code>negative_product_target</code>)
<code>fact_performance</code>	raw metrics stamped <code>snapshot_date</code> — every upload = one snapshot

Two golden schema rules. (1) **Never store derived rates** — ACoS/CTR/CVR/ROAS/break-even/profit are all computed in `metrics.py` / pipeline modules from raw counts. (2) **FKs use ON DELETE CASCADE**, SQLite FK enforcement ON.

Side tables (deliberately separate from the star schema)

Several features keep their own tables so each runs off its own dedicated upload and never shifts the audit snapshot:

Table(s)	Feature
<code>fact_sales</code> , <code>fact_daily</code>	Sales; Monitoring daily tracker (one row/day, upserted by date)
<code>fact_placement</code>	Placement optimizer
<code>daily_watch_fact</code> , <code>watched_campaign</code>	Daily Watch spike tracker (own time series)
<code>weekly_term_fact</code>	Weekly Optimization (STR-driven, real IDs)
<code>mid_month_term_fact</code>	Mid-Month Check
<code>full_month_term_fact</code>	Full Month Audit (in base db)
<code>pause_scale_term_fact</code>	Pause/Scale Audit
<code>product_ad_fact</code>	Product Ads tab
<code>waterfall_run</code> , <code>waterfall_item</code>	Waterfall Restructure Engine

<code>bid_ledger</code>	Effective-bid ledger (double-apply guard)
<code>overlap_finding</code>	Cannibalization detector
<code>sp_channel_fact, sb_fact, sd_fact</code>	Channels (SP/SB/SD account-level)
<code>cadence_run, cadence_task</code>	Cadence run persistence (year, month, audit_type)
<code>product_benchmark</code>	Product Benchmark rows
<code>mined_keyword</code>	Keyword research pool (sources: <code>sqp</code> / <code>cerebro</code> / <code>harvest</code> / <code>ngram</code> / <code>STR</code> / <code>manual</code>)
<code>change_log</code>	Every emitted bulk action logged here
<code>checklist_item</code>	SOP effort checklist per cadence
<code>month_sales_override</code>	Manual monthly sales override
<code>tracker_project, tracked_competitor, tracked_keyword, rank_snapshot, keyword_usage, listing_copy, banned_keyword</code>	Listing Optimizer (Competitor / SEO tracker)

Amazon entity IDs are 16–18 digits — past float64 exact range. `ingest` reads every `* ID` column as string (`dtype=str`); `clean._idstr` keeps them exact. Reading as numbers mangles big IDs (`...929` → `...929.0`), which Amazon rejects as "temporary ID" or collides into "Duplicate Id".

8. Metrics & Bid Math — `metrics.py`

All rate and bid math is pure functions here — nothing derived is ever persisted.

Function	Formula / role
<code>ctr(clicks, impressions)</code>	$\text{clicks} \div \text{impressions}$
<code>cpc(spend, clicks)</code>	$\text{spend} \div \text{clicks}$
<code>cvr(orders, clicks)</code>	$\text{orders} \div \text{clicks}$
<code>acos(spend, sales)</code>	$\text{spend} \div \text{sales}$
<code>roas(sales, spend)</code>	$\text{sales} \div \text{spend}$
<code>all_metrics(...)</code>	bundles the above from raw counts
<code>target_cpc_bid(clicks, sales, target_acos, ...)</code>	bid so projected ACoS = goal
<code>target_acos_bid(current_bid, observed_acos, target, ...)</code>	scale current bid toward goal ACoS
<code>safe_bid_cut(bid, cpc, acos, ...)</code>	guarded downward bid move

Flag engine — `rules.py`

Flag rules are pure `(metrics, thresholds, ctx) -> Flag | None`, registered in `TARGET_RULES`. Adding a rule = add a function + a list entry.

```
TARGET_RULES = [bleeding, high_acos, scale_winner, wasted_spend, low_ctr, low_cvr, overbid]
```

Flag	Meaning	Action
<code>BLEEDING</code>	Worse than <code>HIGH_ACOS</code>	Cut hard toward goal
<code>HIGH_ACOS</code>	Over goal ACoS	Reduce bid (ladder)
<code>WASTED_SPEND</code>	Spend, no/low orders	Negate / pause
<code>LOW_CTR</code>	Impressions, few clicks	Creative / relevance
<code>LOW_CVR</code>	Clicks, few orders	Listing / bid down
<code>OVERBID</code>	Bid > needed for goal	Trim bid
<code>SCALE_WINNER</code>	Converts $\leq$ goal ACoS	Raise bid / budget

`audit.py` injects per-target context: the over-goal history for a reduce→monitor→pause **bid ladder**, per-ASIN break-even, and per-ASIN goal override.

Point-in-time consistency

Multiple uploads accumulate snapshots. "Current period" views must use the **latest** snapshot, never the lifetime sum:

- `audit.active_snapshot(db)` = the latest `FactPerformance` date. Used by `audit()`, `build_tree()`, `sales._ppc_by_asin()`, `profit.compute()`, `sales_view()`.
- `build_tree` (the hot path — a Python loop over all dims+facts) is **cached** per `(db url, snapshot, max perf_id)` and invalidates automatically when fact rows change.
- **Account ad cost** = Σ **Spend of Entity=campaign rows** (`sales.total_ad`), NOT an ASIN rollup — a rollup double-counts campaigns advertising multiple ASINs.

9. Pipeline Stages & Flag Engine

The pipeline is staged: `ingest` → `split` → `clean` → `load`, then `audit`. Around that spine sit many side pipelines.

Core stages

Stage	File	Role
Ingest	<code>ingest.py</code>	Read workbook (via <code>workbook.py</code>), all * <code>ID</code> cols as string, auto-pick sheet, cache by (path,size,mtime)
Split	<code>split.py*</code>	Separate entity types (campaign / ad group / keyword / product target / negatives)
Clean	<code>clean.py</code>	Strip \$/%, <code>_idstr</code> keeps IDs exact, normalize columns
Load	<code>load.py</code>	Upsert dims + insert facts; store existing negatives as DimTarget; defer FKs
Audit	<code>audit.py</code>	Run flag engine over targets with injected context

(*split logic lives inside ingest/clean flow.)

Side pipelines

Module	Does
<code>harvest.py</code>	Search-term harvest: winners→Exact keyword/PT, losers→negate; Amazon-safety rules (ASIN↔keyword separation, no kw+pt mix per ad group, no auto-clause promote, in-file dedup)
<code>ngrams.py</code>	N-gram mining over search terms
<code>trends.py</code>	Snapshot-over-snapshot trends
<code>sales.py</code>	Business-report sales; PPC-by-ASIN; account totals
<code>profit.py</code>	P&L: ad cost + COGS + Amazon fees + referral/refund → net; break-even ACoS
<code>benchmark.py</code>	Product Benchmark match; <code>_unit_economics</code> break-even math
<code>bidopt.py</code> / <code>bid_optimizer.py</code>	Bid plan with guardrails (step cap, hard caps, no thin-data raise)
<code>placement.py</code>	Placement (top-of-search / rest / product page) optimizer
<code>strategy.py</code> / <code>tiers.py</code>	Strategy advisor + Strategy Tier Router

<code>keywords.py</code>	Keyword research pool (SQP / Cerebro / harvest / ngram / STR)
<code>report.py</code>	Exec report + exports
<code>checklist.py</code>	SOP effort checklist
<code>changelog.py</code>	Log every emitted bulk action
<code>monitoring.py</code>	Daily SALES & PPC tracker (FactDaily), health score, alerts
<code>automate.py</code>	One-click combined bulk emit
<code>dashboard.py</code>	ASIN forest + analytics for Dashboard

ASIN-rooted view

Storage is campaign-rooted; `build_tree` re-roots at query time by walking `ad` → `ad_group` → `campaign`, grouping by the ad's ASIN. Campaign/ad-group metrics are the whole campaign (shared up under each ASIN); ad/target level is exact. `/dashboard` returns the full forest as `trees` (every ASIN node, ordered by spend); the frontend `AsinTree` renders each ASIN as a collapsible row.

10. Audit Cadences (5 workflows)

A cadence is a recurring audit workflow. Presets live in `cadence.py`; each sets a lookback label, `Thresholds` overrides that re-tune the flag engine, a SOP checklist, a `focus` flag set, and a dedicated feature/table. **Strict isolation:** each cadence has its own db file and its own upload — data and refetches must never cross.

Cadence	Module	Own table	Feature
Daily Watch	<code>dailywatch.py</code>	<code>daily_watch_fact</code>	day-over-day anomaly tracker (watch-only, no bulk)
Weekly Optimization	<code>weekly.py</code>	<code>weekly_term_fact</code>	SP STR bid tweaks + harvest, per-week panels
Mid-Month Check	<code>midmonth.py</code>	<code>mid_month_term_fact</code>	bid adjust + heavy negatives + bleeders
Full Month Audit	<code>fullmonth.py</code>	<code>full_month_term_fact</code>	superset: bid_tweaks + promotes + negates + bleeders
Pause/Scale	<code>pausescale.py</code>	<code>pause_scale_term_fact</code>	acts on existing entities: scale / pause target / pause campaign

Shared STR engine

The search-term compute is model-agnostic so cadences reuse it on their own tables: `compute_bid_tweaks(rows, t)`, `compute_harvest(rows, t, min_orders, bleeder_acos_mult)`, `summarize(rows)`, `to_bulk`, `parse_str_sheet`, `compare_rows(prev, cur, target_acos)`. Weekly is the reference implementation; Mid-Month adds bleeders via `bleeder_acos_mult=2.0`; Full-Month runs all lists at full 30-day thresholds; Pause/Scale uses `aggregate_targets(rows)` and acts on existing IDs.

Detail per cadence

Daily Watch (`dailywatch.py`)

Own table `DailywatchFact` (one row/campaign/day, upserted by `(date, campaign_id)`), deliberately separate from `FactPerformance` so daily spike-tracking never shifts the audit's latest snapshot. Each upload parses bulk campaign rows (`ingest→split→clean`, then reads `frames["campaign"]` directly — not `load.load`) and replaces that day. `compare(prev, cur)` diffs account + per-campaign rows for spike flags (spend spike, ACoS spike, sales drop, zero-order spend); `series` returns accumulated daily totals for the trend chart.

Weekly (`weekly.py`) — SP only

Driven straight from the bulk **SP Search Term Report** sheet. Own table `WeeklyTermFact` carries real Campaign/Ad Group/Keyword/PT IDs as exact strings. Mirrors the Daily Watch calendar grid but panels = weeks (min 5); each week uploads its own bulk (upsert per week), weeks accumulate into `series()`, `plan(db, t, week)` scopes to the selected week (default = latest via `_resolve_week`). Legacy single-snapshot tables auto-migrate. `weekly.compare` = week-over-week.

Mid-Month (`midmonth.py`)

Single-panel, focused on bid adjustments + heavy negative targeting. Calls the shared engine with `bleeder_acos_mult=2.0` to return a third list, **bleeders** (converted but ACoS $\geq 2\times$ goal), pre-filtered to act-worthy (`bleeder_min_orders=2`, spend $\geq$ `bleeder_spend_mult(2.0)`×loser-floor). Bleeders are NOT pre-selected (negating them stops sales). On each upload the prior snapshot shifts to `period=1` so "Compare to previous" works.

Full Month (`fullmonth.py`)

Superset — returns `bid_tweaks` + `promotes` + `negates` + `bleeders` at once (full 30-day thresholds). Same previous-upload history as Mid-Month.

Pause/Scale (`pausescale.py`)

Unlike the others, acts on **existing entities** (not customer search terms): `scales` (10+ orders $\leq$ goal $\rightarrow$ bid up), `pauses` (30+ clicks, 0 orders $\rightarrow$ paused), `campaign_pauses` (\$50+ spend, 0 orders $\rightarrow$ pause whole campaign). Its `to_bulk` emits Keyword/PT update + Campaign update by exact ID; logged via `changelog.from_pausescale`.

Frontend gating: all five cadences hide the generic flag table in favor of their dedicated panels. The App gate is `!['watch', 'weekly', 'mid_month', 'full_month', 'pause_scale'].includes(feature)`. Cadence runs persist per (year, month, audit_type) in `CadenceRun/CadenceTask`. Per-cadence Strategy Advisor is `cadstrat.py` (drives strategies from each cadence's own side-table, not the star schema).

11. Machine Learning Module — `ml.py`

NumPy-only ML for the cadences: no new deps, no persistence, no RNG — deterministic per plan. Advisory only; it never changes exported bulk bids.

Technique	Function(s)	Used by
Empirical-Bayes Beta-binomial CVR shrinkage	<code>fit_beta_prior</code> , <code>posterior</code> , <code>p_below</code>	fixes thin-data CVR (own <code>betainc</code> continued-fraction, no scipy)
Robust anomaly detection (median/MAD z-score)	<code>robust_anomalies</code>	Daily Watch
Double exponential smoothing	<code>holt_forecast</code>	Weekly next-week projection
Hand-rolled logistic regression (AUC-gated)	<code>LogReg</code> , <code>train_term_model</code> , <code>score_terms</code>	Full Month Early Promote Candidates

`enrich_plan(plan, agg_rows, target_acos)` is the integrator every cadence calls at the end of its `plan()`. It stamps each row with `row["ml"]` (smoothed CVR + confidence) and sets `plan["ml"]` (prior + break-even CVR). When the account is too small, every model degrades silently to `None` — never fabricates. Frontend: `MLInsights.jsx` per-cadence card + `mlCols/withML()`.

12. Account-Level Engines

Newer engines are account-level: always the base project db (via `get_base_db`, cadence-agnostic like Monitoring), own tables, never touch `FactPerformance/active_snapshot`. All IDs are exact strings.

Waterfall Restructure Engine — `waterfall.py`

One uploaded bulk → a draft restructure plan into the RAF Funnel System (AT discovery → KWT-BROAD → KWT-PHRASE → KWT-EXACT money tier + PT ASIN tier; each downstream promotion emits a negativeExact upstream; 1 SKU per campaign; a boss campaign per SKU+slot; duplicates consolidated).

The plan is **4 phased bulks in dependency order**:

Phase	Content	Risk
A	Renames	zero risk
B	Loser bid cuts	transitional overlap, not a pause
C	Creates (new campaigns born paused + seeds + sculpting negatives)	additive
D	Pauses	UI-gated until A–C marked applied (<code>/waterfall/applied</code> enforces order)

Own tables `WaterfallRun/WaterfallItem`. Settings via `get_project_extra`: naming template, budget, seeds, heroes, `bid_floor/bid_ceiling`, `force_down_only`, `protect_min_orders`, `protected_campaign_ids`, per-slot `slot_bids`, `new_strategy`. `render_name` placeholders: `{sku}/{asin}/{slot}/{n}/{bid}/{strategy}`. **Boss override**: `select_bosses(classified, goal, forced=...)` honors user picks; upload saves the raw bulk so `POST /waterfall/override` re-runs the plan with no re-upload. Phase-D pause protection guards empty campaigns with orders. Day-0 hero benchmark vs latest → day-21 verdict. Frontend `waterfall.jsx`.

Effective-Bid Ledger — `ledger.py`

Fixes: consecutive exports from the same snapshot double-apply bid changes. Every bulk-emitting module `record()`s exported bid/state changes; later bid computations read the **effective** value = latest ledger row else snapshot. Lifecycle: exported (`applied=False`) → "Mark applied" or auto-reconcile on next upload → or "Discard". Unapplied rows older than `STALE_DAYS` (7) raise a warning. Always base db via `base_session(db)`.

Cannibalization Detector — `cannibal.py`

Findings regenerated wholesale per run (idempotent replace). **Type 1** = duplicate targets (same keyword_text+match_type or PT expression enabled in 2+ campaigns). **Type 2** = cross-product STR overlap (same customer search term converting for 2+ SKUs). Owner rule `resolve_owner` (pure): candidates with clicks ≥ min_clicks, max CVR wins, tiebreak min ACoS. 'coexist' verdicts (no action) when all candidates profitable, or same-SKU tier pair (waterfall working). **Converter guard**: never emit a negative where the term itself converts or is a live enabled keyword. Shares the Waterfall upload (`?engines=waterfall,cannibal`). Frontend `Cannibalization.jsx`.

Channels — `channels.py`

SB/SD visibility (v1 read-only, no SB/SD bulk emit). One bulk workbook → three tables: `SPChannelFact`, `SBFact` (from the **"SB Multi Ad Group Campaigns" sheet ONLY** — the legacy sheet duplicates campaigns; ingesting both double-counts), `SDFact`. Report: channel mix cards, SB keyword table with HIGH_ACOS/WASTED_SPEND flags, brand vs non-brand split from the per-store brand-term list, SD dormant banner, read-only SB STR harvest. Frontend `Channels.jsx`.

Consultation / Tier Recommendations — `consult.py`

One SP bulk → count advertised ASINs → route to a 7-tier ladder (`consult.TIERS`) → tier-thresholded problem scan (WASTED_SPEND/HIGH_ACOS/RAISE_WINNER/OVERBID/MIXED_AD_GROUP/HARVEST_CANDIDATE with per-tier resolutions). No own tables — result JSON persists via `set_project_extra` key `consult`. Frontend `Consultation.jsx`.

13. Store-Level: Catalog, Transactions, Listing Optimizer

Product Benchmark Catalog — `catalog.py`

From Amazon **Category Listings Reports**. Store-level `_catalog.json`, shared by every audit/cadence like Product Costs. Amazon exports one `.xlsm` per category → uploads **MERGE by SKU** (upsert, not replace). Sheet "Template": header row of display names, then an attribute-key row (`item_name[...][language_tag=en_US]#1.value`), then an example row, then data — parse keys off the attribute-key row, never column positions (800+ cols drift per category). Frontend `Catalog.jsx`.

SKU Transactions — `transactions.py`

From the Payments **Date Range (transaction) report**. Store-level `_transactions.json`: per-SKU order/refund economics (product sales, selling/FBA fees, promos, net proceeds), date-filtered. Uploads **merge + dedupe row-by-row** — monthly re-uploads accumulate into a continuous ledger. Rows without SKU (Transfer, Service Fee, FBA Inventory Fee...) are account-level, reported separately. Parser skips the quoted preamble to the header starting `date/time`. Frontend `Transactions.jsx`.

FBA / Referral Fees — `fbafees.py`

Base fulfillment fee + referral fee per unit from the **Selling economics (SKU Economics)** report or FBA Fee Preview — one loose-column parser for all. ALL sheets parsed; a Referral Fee Preview sheet yields `kind="referral"` rows that `merge()` applies over the matching item's referral fee and recomputes `total_fee`. `total_source = report / scaled / derived`. Both fees win over ledger-observed values; their sum `total_fee` is the exact per-unit Amazon fee charged by `benchmark._unit_economics`: $BE = (price - COGS - total_fee) \div price$. Mapped to CHILD/standalone ASINs only.

Listing Optimizer — `tracker.py`

Competitor Research / Indexed Keywords / SEO tracker. Base db via `get_base_db`. Tables: `TrackerProject`, `TrackedCompetitor`, `TrackedKeyword`, `RankSnapshot`, `KeywordUsage`, `ListingCopy`, `BannedKeyword`. Replaces the "Project Snore" Google Sheet — the approved mapping `research_tracker/MAPPING.md` wins on semantics. Two entry flows: one-time `migrate` sheet import, or a raw flow (blank project + Cerebro/X-ray imports + pasted listing copy). Views (`matrix/scorecard/movers/suggest/export_matrix`) recomputed live.

Rank semantics: integer = organic rank; `'-'/0` = checked-but-unranked (stored as a `None` snapshot row); NO row = unchecked. Snapshots are append-only; re-import of a date replaces that date only.

Keywords ↔ tracker integration: `tracker.add_keywords` dedupe-merges into `TrackedKeyword` (sources `sqp/cerebro/harvest/ngram/manual`). `/keywords/to-project` reads `MinedKeyword` from the cadence db and writes via `ledger.base_session.tracker.relevancy_prompt` = a copyable LLM prompt (keywords vs CURRENT+PROPOSED `ListingCopy`). `MinedKeyword` has a third source **STR**; `/harvest/from-bulk` auto-merges its terms via `keywords.merge_str_terms`. `project-preview` = a forecast scorecard incl. Impression Share.

14. Bulk File Safety & Parsers

Bulk output must pass Amazon validation — `bulkfmt.py`

Any engine emitting an Amazon bulk file (`automate`, `bidopt`, `placement`, `harvest`, `strategy`, `cadences`, `waterfall`) logs actions to `ChangeLog` and routes output through `bulkfmt.py`:

- ASIN search terms become `asin="B0..."` **product targets** (negative), never keywords.
- Auto-campaign clauses (loose/close-match, substitutes, complements) can't be negated → skipped.
- Keyword text checked with `bulkfmt.valid_keyword` (≤80 chars, ≤10 words, no control/special chars) or dropped ("Keyword is invalid").
- IDs emitted via `bulkfmt.idstr` as exact integer strings.
- A target repeated in-file (update **or** create/negate) or already in the account is dropped (no "Duplicate Id" / "Duplicate Keyword Text" / "already exists").

To know what already exists, `load.py` stores existing **Negative Keyword / Negative Product Targeting** rows as `DimTarget`. These carry no facts and are excluded from `audit/bidopt/build_tree`. Existing data must be re-uploaded once for account-dedup to apply.

Tolerant parsers

Amazon/Helium10 exports drift, so parsers (`ingest`, `harvest.parse_str`, `sales.parse_business_report`, `benchmark.parse_benchmark`, `keywords`) accept `.xlsx/.xlsm/.csv`, auto-pick the right sheet, skip metadata-preamble rows to find the real header (Brand Analytics SQP), strip `$/%`, handle 7/14-day column drift, and convert ROAS↔ACoS. Add format support by matching columns loosely, never by exact header.

Read engine — `workbook.py`

Reads via **calamine** (Rust) when installed — ~8x faster than `openpyxl` on a big bulk (120k-row SP sheet: ~2.5s vs ~21s). Identical `dtype=str` output so 16–18 digit IDs stay exact. `workbook.excel_engine()` resolves once at import and **falls back to openpyxl** if absent. `catalog/transactions/fbafees` use `openpyxl` `read_only=True` streaming (already lean). Reads cached by (path,size,mtime). **Writes** (13 `ExcelWriter` sites) stay on `openpyxl`.

15. API Reference

All data routers are mounted in `main.py` gated by `Depends(auth.current_user)`. Only `/auth/*` and `/health` are public. Every data call also carries `?store=&project=&audit_type=`.

Auth & Stores

Method + Path	Purpose
POST <code>/auth/register</code> · login · logout · GET <code>/auth/me</code>	Session lifecycle
GET POST <code>/auth/users</code> · DELETE <code>/auth/users/{u}</code>	Superuser user CRUD
GET POST <code>/stores</code> · GET <code>/stores/overview</code>	Stores + cross-store KPI rollup
GET POST <code>/projects</code>	Audits within a store

Pipeline / Audit / Optimize

Method + Path	Purpose
POST <code>/upload</code>	Ingest a bulk / export → build star schema
GET <code>/audit</code> · <code>/asins</code> · <code>/asins/{asin}/tree</code>	Flags + ASIN summaries + tree
GET <code>/dashboard</code> · <code>/dashboard/analytics</code> · <code>/dashboard/export</code>	ASIN forest + analytics
GET <code>/bids/optimize</code> · POST <code>/bids/optimize/bulk</code>	Bid optimizer + bulk
GET <code>/placements</code> · POST <code>/placements/bulk</code>	Placement optimizer + bulk
POST <code>/harvest</code> · <code>/harvest/bulk</code> · <code>/harvest/from-bulk[/file]</code>	Search-term harvest
POST <code>/ngrams</code> · <code>/ngrams/export</code> · GET <code>/trends</code>	N-gram mining + trends
GET <code>/strategy</code> · <code>/strategy/tier</code> · POST <code>/strategy/bulk</code>	Strategy advisor + tier + bulk
POST <code>/automate</code>	Combined one-click bulk emit
GET <code>/report</code> · <code>/report/summary</code> · <code>/report/export</code>	Exec report + export
POST <code>/narrate</code> · GET <code>/config</code>	Optional LLM narration + config

Cadences

Method + Path	Purpose
GET <code>/cadence/presets</code> · GET POST <code>/cadence/runs</code> · DELETE <code>/cadence/runs/{id}</code>	Presets + run persistence

POST /daily-watch/upload · GET /daily-watch/days compare series anomalies · monitors	Daily Watch
POST /weekly/upload · GET /weekly/weeks series compare plan · POST /weekly/bulk	Weekly
POST /mid-month/upload · GET /mid-month/plan compare · POST /mid-month/bulk	Mid-Month
POST /full-month/upload · GET /full-month/plan compare · POST /full-month/bulk	Full Month
POST /pause-scale/upload · GET /pause-scale/plan · POST /pause-scale/bulk	Pause/Scale

Account engines

Method + Path	Purpose
GET PUT /waterfall/settings · POST /waterfall/upload · GET /waterfall/run · POST /waterfall/bulk applied select override	Waterfall
GET /waterfall/benchmark · /waterfall/ledger · POST /waterfall/ledger/applied discard	Benchmark + ledger
POST /cannibal/run · GET /cannibal/findings summary · POST /cannibal/bulk	Cannibalization
POST /channels/upload · GET /channels/summary sb-keywords sd-targets sb-harvest · GET PUT /channels/brand-terms	Channels
GET /consult/tiers · POST /consult/upload · GET /consult/run	Consultation

Store-level, Keywords, Tracker, Product Ads, Monitoring

Method + Path	Purpose
POST /catalog/upload · GET /catalog · /catalog/item export · PUT /catalog/cogs	Product catalog
POST /catalog/fba-fees/upload · GET /catalog/fba-fees	FBA/referral fees
POST /catalog/transactions/upload · GET /catalog/transactions[/export]	SKU transactions
POST /keywords/upload · GET /keywords recommend backend-terms · POST /keywords/to-project export · GET POST /keywords/project-preview	Keyword research

GET POST /tracker/projects · POST /tracker/migrate import xray · GET /tracker/matrix scorecard movers seo-recommend · GET PUT /tracker/listing banned · POST /tracker/keywords/bulk · GET /tracker/relevancy-prompt	Listing Optimizer
POST /product-ads/upload · GET /product-ads[/detail export]	Product Ads
GET /benchmark · POST /benchmark/upload	Product Benchmark
(monitoring routes) GET summary over date range	Daily SALES & PPC tracker
GET /templates · /templates/{kind}	Blank/reference template files

Interactive OpenAPI docs are served at </docs> when the API runs.

16. Frontend Architecture

`App.jsx` is a tabbed shell (sidebar nav, `TABS` const).

Tabs

Dashboard · Stores · PPC Audit · Product Ads · Waterfall · Cannibalization · Channels · Monitoring · Keywords · SEO · Listing Audit · Product Overview · Product Benchmark (catalog + transactions) · Transactions · Strategy · Reports · Change Log. `CADENCE_TABS` (`ppc`, `strategy`) append the cadence slug to the URL path.

Data flow & scoping

- `api/client.js` holds module-global `STORE/PROJECT/CADENCE`; `q()` injects `?store=&project=&audit_type=` into every call.
- Components fetch on a `scope` prop `${store}:${project}:${dataVer}`; `dataVer` is bumped after any edit to force a refetch.
- `App` calls `api.setCadence()` **synchronously** when a cadence is picked (before child panels refetch); per-cadence panels carry `auditType` in the `scope` prop so they refetch on cadence switch.
- Ephemeral results that must survive tab switches (harvest / n-gram) are lifted to `App` state and passed down; components fall back to local state when used standalone.

One table everywhere — `components/table.jsx`

`useTableControls` (search + composite/per-column filters + sort + page-size + pagination) powers a declarative `DataTable({ columns, rows, rowKey, selectable, selected, onToggle, ... })`. Each column is `{ key, label, align?, render?(row), value?(row), sortKey?, cellClass? }` where `render` is cell display and `value/row[key]` is what sort/filter use. All 16 data tables use it (cadence panels via `DataTable`). A `lean` prop drops the toolbar/pagination for small tables and auto-falls-back to the full toolbar past `leanRows` (default 12). Shared cadence column defs live in `components/cadenceCols.jsx`.

Modals — `components/Modal.jsx`

`<Modal>` = base portal shell. `<ModalProvider>` (mounted once in `App`) exposes `useModals()` → promise-based `confirm` / `prompt` / `alert` (near drop-ins for the native dialogs). `danger:true` styles the action red + disables backdrop-dismiss. **Never reintroduce** `window.confirm/prompt/alert` — route through `useModals()`.

17. Design System

Binance-style, token-driven so the whole app reskins centrally. **Never hardcode hex** — use Tailwind tokens in `tailwind.config.js`.

Token	Hex	Role
ink	#0b0e11	canvas
panel	#1e2329	surface
edge	#2b3139	hairline / elevated
mute	#707a8a	muted text
body	#eaecef	body text
primary / lime	#FCD535	Binance Yellow — single brand accent
primaryActive	#f0b90b	active/pressed accent
onPrimary	#000000	text on yellow (black)
up / green	#0ecb81	positive values (text color)
down / red	#f6465d	negative values (text color)

Shared utilities in `index.css`: `.card` (surface + hairline + 12px), `.btn/.btn-primary` (yellow+black), `.btn-ghost`, `.tag`. Fonts: `font-sans` = Inter (copy/UI), `font-mono` = JetBrains Mono (all numbers). Flat color-block depth — no shadows/gradients/grid. Trading semantics: green/red only as **text color** for +/- values, not button fills.

Sticky headers are global via `index.css`: under fine-pointer media, `div:has(> table)` becomes a `max-height:70vh; overflow:auto` scroll box and its `thead th` is `position:sticky; top:0` — any table that is a direct child of a `.overflow-x-auto` div gets sticky headers free.

18. Development, Testing & Conventions

Commands

```
# Backend (from backend/)
source .venv/bin/activate
uvicorn app.main:app --port 8000      # serve API (docs at /docs)
python -m app.main                    # (re)seed bundled ZValves sample
python -m pytest -q                   # run tests
python -m pytest tests/test_core.py::test_wasted_spend_flag # single test

# Frontend (from frontend/)
npm run dev      # http://localhost:5173, proxies /api -> :8000
npm run build    # ALWAYS run after frontend edits to verify JSX compiles
```

No lint step. After a backend change run `pytest`; after a frontend change run `npm run build` (catches JSX/import errors).

Test suites (backend/tests/)

`test_core`, `test_dashboard`, `test_goal_acos`, `test_overbid`, `test_ml`, `test_tiers`, `test_waterfall`, `test_cannibal`, `test_channels`, `test_consult`, `test_catalog`, `test_transactions`, `test_fba_fees`, `test_harvest_from_bulk`, `test_harvest_scope`, `test_keywords_preview`, `test_tracker`, `test_tracker_keywords`, `test_dailywatch_monitor`, `test_cadence_isolation`, `test_cadence_clear`, `test_cadence_run_race`. Test suites build synthetic bulks in-test; `files/bulk-*.xlsx` are real exports for manual regression.

Optional LLM — `llm/`

Provider-agnostic (Ollama / LM Studio / OpenAI-compatible), default `none`. All bid/profit math is deterministic — the LLM only narrates. The app is fully functional without it.

Golden conventions (from CLAUDE.md)

- Never store derived rates — compute in `metrics.py`.
- Never let an entity ID become a float — read as string end to end.
- FKs cascade; SQLite FK enforcement ON.
- Current-period views use the latest snapshot, never the lifetime sum.
- All emitted bulks route through `bulkfmt.py` and log to `ChangeLog`.
- Strict per-cadence isolation — data + refetches never cross cadences.
- Update `MANUAL.md` with every feature change.
- Add CSV/format support by matching columns loosely, never by exact header.
- Route all confirms through `useModals()`, never native dialogs.
- Never hardcode hex — use Tailwind tokens.

— End of documentation —